

## Unidad 2: Distintos modelos NoSQL

### Clase 3 – Modelo documental y consultas NoSQL: MongoDB como eje práctico

#### Objetivo

*Diseñar y consultar estructuras documentales utilizando MongoDB, comprendiendo el modelo documental, la desnormalización y las operaciones básicas de acceso y recuperación de información.*

#### Tema

- Modelos NoSQL: visión general y comparativa.
- Introducción al modelo documental.
- MongoDB / MQL:
  - Estructura documental
  - CRUD básico
  - Consultas simples
  - Escalabilidad y desnormalización.

#### Introducción breve al modelo documental y JSON

Dentro del universo NoSQL existen varios modelos de bases de datos, entre ellos clave–valor, columnas anchas, grafos y documentales. En esta clase nos enfocamos en el modelo documental porque es uno de los más utilizados cuando se necesita trabajar con información flexible, cambiante y cercana a la forma en que las aplicaciones modernas intercambian datos.

El modelo documental organiza la información en **documentos** en lugar de tablas y filas. Un documento reúne en una sola estructura distintos datos relacionados entre sí. En MongoDB, esos registros se almacenan como **documentos BSON**, que es una representación binaria de JSON con tipos de datos adicionales. Esto significa que, aunque para nosotros el documento se vea parecido a JSON, internamente MongoDB lo guarda en un formato optimizado para almacenamiento y recorrido eficiente de los datos.

Para entender el modelo documental, primero hay que entender qué es **JSON**. JSON, o JavaScript Object Notation, es una forma simple y legible de representar información mediante pares clave–valor. Una clave identifica un dato y el valor contiene su contenido. Además, JSON permite agrupar datos dentro de objetos y arreglos, por lo que una misma estructura puede contener texto, números, listas y objetos anidados. Justamente esa capacidad de anidamiento es la que vuelve tan natural al modelo documental para representar entidades de negocio complejas.

Por ejemplo, en un modelo relacional un pedido podría requerir varias tablas: una para el pedido, otra para el cliente y otra para los productos. En cambio, en un modelo documental ese mismo pedido puede quedar representado en un único documento, incluyendo al cliente y la lista de productos dentro de la misma estructura. Así, el dato se guarda de una forma más cercana al uso real que hará la aplicación.

```
{
  "pedidoId": 1001,
  "cliente": {
    "nombre": "Ana López",
    "ciudad": "Morón"
  },
  "productos": [
    { "nombre": "Teclado", "cantidad": 1, "precio": 25000 },
    { "nombre": "Mouse", "cantidad": 2, "precio": 12000 }
  ],
  "estado": "en preparación"
}
```

Este ejemplo muestra una idea clave del modelo documental: un documento puede contener **objetos embebidos y arreglos**. MongoDB admite precisamente estas estructuras para representar relaciones entre datos dentro del mismo documento. Esto permite resolver muchos casos de uso sin necesidad de hacer uniones como en SQL y abre la puerta a la **desnormalización**, es decir, guardar juntos datos que en una base relacional normalmente estarían separados.

La desnormalización no significa desorden. Significa que el diseño se piensa según cómo se va a consultar la información. Si en el negocio suele recuperarse el pedido junto con sus productos y datos básicos del cliente, entonces tiene sentido mantenerlos integrados en un mismo documento. De esa manera, el acceso resulta más directo y el modelo acompaña mejor el flujo real de lectura de datos. Las bases documentales se destacan justamente por ofrecer un esquema flexible y por permitir que el modelo evolucione a medida que cambian los requerimientos de la aplicación.

Ahora bien, aunque en clase hablemos de JSON porque es legible y fácil de comprender, en MongoDB el formato real de almacenamiento es BSON. La diferencia es importante. JSON es ideal para que las personas lean y escriban estructuras de datos; BSON, en cambio, agrega tipos útiles para una base de datos, como fechas, datos binarios, enteros de distintos tamaños y ObjectId, además de facilitar el procesamiento interno. Por eso, cuando se vea un documento en pantalla, se interpretará como JSON, pero técnicamente MongoDB estará trabajando con BSON.

En el modelo documental los datos no se piensan como filas aisladas sino como **unidades de información completas**; JSON es la forma más intuitiva de escribir esas estructuras; y MongoDB toma esa lógica y la implementa mediante documentos BSON, con flexibilidad de esquema, posibilidad de anidar información y una organización más cercana a muchos problemas reales del negocio.

## MongoDB

La consola mongosh es el entorno interactivo donde trabajás directamente con MongoDB. Podés pensarla como una terminal inteligente para manipular bases documentales. Desde ahí creás bases, insertás documentos JSON, consultás información, actualizás datos y explorás la estructura completa sin necesidad de una interfaz gráfica.

Cuando abrís mongosh, lo primero que ves es algo como `test>` o el nombre de la base actual. Ese texto indica en qué base estás trabajando. Todo lo que hagas se ejecuta dentro de ese contexto. Si querés cambiar de base, usás el comando `use`. Esto no crea una base inmediatamente, solo cambia el contexto y la base se materializa cuando insertás datos.

Dentro de la consola tenés varios tipos de comandos. Algunos sirven para navegar, otros para manipular datos y otros para inspeccionar el entorno. Por ejemplo, `show dbs` te permite ver las bases existentes, `show collections` muestra las colecciones de la base actual, y `db` te dice en qué base estás parado.

Una característica importante es que la consola entiende directamente objetos JSON. Eso significa que no escribís SQL, sino estructuras clave–valor. MongoDB trabaja con documentos, y mongosh te permite escribirlos de forma natural. Esto hace que el aprendizaje sea más intuitivo porque los estudiantes ven directamente la estructura de los datos.

También es útil saber cómo funciona el prompt. Cuando ves `>`, la consola espera un comando nuevo. Cuando ves `|`, significa que la instrucción quedó abierta y la consola está esperando que cierres paréntesis o llaves. Si esto ocurre, podés completar la instrucción o presionar `Ctrl+C` para cancelarla.

Desde la consola podés hacer prácticamente todo lo necesario para trabajar con MongoDB. Podés crear bases, crear colecciones, insertar documentos, consultar con filtros, actualizar campos y eliminar registros. Todo esto sin necesidad de definir estructura previa, lo cual refuerza el concepto de esquema flexible.